Network Resource Allocation

Problem statement -

In computer Network, multiple users or applications need resources such as bandwidth & network channels. These resources are limited, so we must allocate them efficiently

- The goal is to find the best allocation of network resources so that network performance is maximized & resource wastage is minimized

Pseudo code

1. Initialize a random population of resource allocations.
2. Repeat until the termination condition is satisfied:
   a. Calculate the fitness of each allocation
   b. Select the best allocations as parents
   c. Perform crossover to generate new allocations
   d. Apply mutation to some new allocations
   e. Check whether each allocation satisfies the
   f. available network resource constraints
   g. Form the new population
3. Select the allocation with highest fitness.
4. Display the best network resource allocation

END.

```python
import random

# Network resource allocation using Genetic Algorithm
TOTAL_BANDWIDTH = 100
DEMANDS = [40, 50, 30]
POPULATION_SIZE = 6
GENERATIONS = 20
MUTATION_RATE = 0.1

# Generate a random allocation that is immediately valid
def create_individual():
    while True:
        individual = [random.randint(0, d) for d in DEMANDS]
        if sum(individual) <= TOTAL_BANDWIDTH:
            return individual
```

```python
# Fitness function: returns the total allocation, or 0 if it violates constraints
def fitness(individual):
    allocated = sum(individual)
    if allocated > TOTAL_BANDWIDTH:
        return 0
    return allocated

# Selection: choose the best two individuals
def selection(population):
    # Sort a copy to avoid unexpected behavior from in-place sorting
    sorted_pop = sorted(population, key=fitness, reverse=True)
    return sorted_pop[:2]

# Crossover: combine two parents
def crossover(parent1, parent2):
    point = random.randint(1, len(DEMANDS) - 1)
    child = parent1[:point] + parent2[point:]
    return child
```

```python
# Mutation: randomly change allocation
def mutation(individual):
    mutated = individual[:]
    for i in range(len(mutated)):
        if random.random() < MUTATION_RATE:
            mutated[i] = random.randint(0, DEMANDS[i])
    return mutated

# Initialize a valid population
population = [create_individual() for _ in range(POPULATION_SIZE)]

# Run Genetic Algorithm
for generation in range(GENERATIONS):
    parents = selection(population)
    new_population = parents[:]

    # Counter to prevent infinite loops if valid children are hard to find
    attempts = 0
    while len(new_population) < POPULATION_SIZE and attempts < 100:
```

```
# Counter to prevent infinite loops if valid children are hard to find
attempts = 0
while len(new_population) < POPULATION_SIZE and attempts < 100:
    parent1, parent2 = random.sample(parents, 2)
    child = crossover(parent1, parent2)
    child = mutation(child)

    if sum(child) <= TOTAL_BANDWIDTH:
        new_population.append(child)
    attempts += 1

# If the loop struggled to find valid mutations, fill the rest with new valid individuals
while len(new_population) < POPULATION_SIZE:
    new_population.append(create_individual())

population = new_population
```

```python
# Find best allocation
best = max(population, key=fitness)

print("Network Resource Allocation")
for i, amount in enumerate(best):
    print(f"User {i + 1}: {amount} Mbps")

print("Total Bandwidth Allocated:", sum(best), "Mbps")
print("Available Bandwidth:", TOTAL_BANDWIDTH, "Mbps")
print("Fitness Value:", fitness(best))
```

```
Network Resource Allocation
User 1: 29 Mbps
User 2: 50 Mbps
User 3: 21 Mbps
Total Bandwidth Allocated: 100 Mbps
Available Bandwidth: 100 Mbps
Fitness Value: 100
```